

Create a Django 5 + Django REST Framework project called "AI-Assisted Box Selection System". Build Product, Box, Order, and OrderItem models, implement a BoxSelector service that recommends the cheapest valid shipping box based on dimensions, volume, and weight, create a POST /api/recommend-box/ API endpoint, add Swagger documentation using drf-spectacular, register models in Django Admin, and write pytest test cases for box selection scenarios. create this project with the given requirement



Below is a complete Django 5 + Django REST Framework project structure for an **AI-Assisted Box Selection System** that satisfies your requirements:

- Django 5
- Django REST Framework
- Product, Box, Order, OrderItem models
- BoxSelector recommendation service
- POST /api/recommend-box/
- Swagger/OpenAPI with drf-spectacular
- Django Admin registration
- Pytest test cases

Project Structure

```
ai_box_selection/
|
├── manage.py
├── pytest.ini
|
├── ai_box_selection/
|   ├── __init__.py
|   ├── settings.py
|   ├── urls.py
|   ├── asgi.py
|   └── wsgi.py
|
└── shipping/
    ├── __init__.py
    └── admin.py
```

```
├─ apps.py
├─ models.py
├─ serializers.py
├─ views.py
├─ services.py
├─ urls.py
├─
└─ tests/
    ├─ __init__.py
    └─ test_box_selector.py
```

Installation

Bash

```
pip install django djangorestframework drf-spectacular
pip install pytest pytest-django
```

settings.py

Python

Run

```
INSTALLED_APPS = [
    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.contenttypes",
    "django.contrib.sessions",
    "django.contrib.messages",
    "django.contrib.staticfiles",

    "rest_framework",
    "drf_spectacular",

    "shipping",
]

REST_FRAMEWORK = {
    "DEFAULT_SCHEMA_CLASS":
        "drf_spectacular.openapi.AutoSchema",
}

SPECTACULAR_SETTINGS = {
    "TITLE": "AI-Assisted Box Selection API",
    "DESCRIPTION": "Box recommendation service",
```

```
"VERSION": "1.0.0",  
}
```

models.py

Python

Run

```
from django.db import models  
  
class Product(models.Model):  
    name = models.CharField(max_length=255)  
  
    length = models.FloatField()  
    width = models.FloatField()  
    height = models.FloatField()  
  
    weight = models.FloatField()  
  
    def volume(self):  
        return self.length * self.width * self.height  
  
    def __str__(self):  
        return self.name  
  
class Box(models.Model):  
    name = models.CharField(max_length=255)  
  
    length = models.FloatField()  
    width = models.FloatField()  
    height = models.FloatField()  
  
    max_weight = models.FloatField()  
    price = models.DecimalField(  
        max_digits=10,  
        decimal_places=2  
    )  
  
    def volume(self):  
        return self.length * self.width * self.height  
  
    def __str__(self):  
        return self.name  
  
class Order(models.Model):  
    created_at = models.DateTimeField(auto_now_add=True)
```

```

def __str__(self):
    return f"Order #{self.id}"

class OrderItem(models.Model):
    order = models.ForeignKey(
        Order,
        on_delete=models.CASCADE,
        related_name="items"
    )

    product = models.ForeignKey(
        Product,
        on_delete=models.CASCADE
    )

    quantity = models.PositiveIntegerField(default=1)

    def __str__(self):
        return f"{self.product.name} x {self.quantity}"

```

services.py

Python

Run

```

from decimal import Decimal

from .models import Box

class BoxSelector:

    @staticmethod
    def recommend_box(length, width, height, weight):

        item_volume = length * width * height

        valid_boxes = []

        for box in Box.objects.all():

            box_volume = (
                box.length *
                box.width *
                box.height
            )

            dimension_fit = (
                length <= box.length and

```

```

        width <= box.width and
        height <= box.height
    )

    volume_fit = item_volume <= box_volume

    weight_fit = weight <= box.max_weight

    if dimension_fit and volume_fit and weight_fit:
        valid_boxes.append(box)

if not valid_boxes:
    return None

return sorted(
    valid_boxes,
    key=lambda b: (
        Decimal(str(b.price)),
        b.volume()
    )
)[0]

```

serializers.py

Python

Run

```

from rest_framework import serializers

class BoxRecommendationRequestSerializer(
    serializers.Serializer
):
    length = serializers.FloatField()
    width = serializers.FloatField()
    height = serializers.FloatField()
    weight = serializers.FloatField()

class BoxRecommendationResponseSerializer(
    serializers.Serializer
):
    box_name = serializers.CharField()
    price = serializers.DecimalField(
        max_digits=10,
        decimal_places=2
    )

```

views.py

Python

Run

```
from drf_spectacular.utils import extend_schema
from rest_framework.response import Response
from rest_framework.views import APIView
from rest_framework import status

from .serializers import (
    BoxRecommendationRequestSerializer,
    BoxRecommendationResponseSerializer
)

from .services import BoxSelector

class RecommendBoxAPIView(APIView):

    @extend_schema(
        request=BoxRecommendationRequestSerializer,
        responses={
            200: BoxRecommendationResponseSerializer
        }
    )
    def post(self, request):

        serializer = (
            BoxRecommendationRequestSerializer(
                data=request.data
            )
        )

        serializer.is_valid(
            raise_exception=True
        )

        box = BoxSelector.recommend_box(
            length=serializer.validated_data["length"],
            width=serializer.validated_data["width"],
            height=serializer.validated_data["height"],
            weight=serializer.validated_data["weight"]
        )

        if not box:
            return Response(
                {
                    "message":
                        "No suitable box found."
                },
                status=status.HTTP_404_NOT_FOUND
            )
```

```
    )

    return Response(
        {
            "box_name": box.name,
            "price": box.price
        }
    )
```

shipping/urls.py

Python

Run

```
from django.urls import path

from .views import RecommendBoxAPIView

urlpatterns = [
    path(
        "recommend-box/",
        RecommendBoxAPIView.as_view(),
        name="recommend-box"
    ),
]
```

project urls.py

Python

Run

```
from django.contrib import admin
from django.urls import path, include

from drf_spectacular.views import (
    SpectacularAPIView,
    SpectacularSwaggerView,
)

urlpatterns = [
    path(
        "admin/",
        admin.site.urls
    ),

    path(
        "api/",
```

```

        include("shipping.urls")
    ),

    path(
        "api/schema/",
        SpectacularAPIView.as_view(),
        name="schema"
    ),

    path(
        "api/docs/",
        SpectacularSwaggerView.as_view(
            url_name="schema"
        ),
        name="swagger-ui"
    ),
]

```

admin.py

Python

Run

```

from django.contrib import admin

from .models import (
    Product,
    Box,
    Order,
    OrderItem
)

@admin.register(Product)
class ProductAdmin(admin.ModelAdmin):
    list_display = (
        "name",
        "weight"
    )

@admin.register(Box)
class BoxAdmin(admin.ModelAdmin):
    list_display = (
        "name",
        "max_weight",
        "price"
    )

```



```

class OrderItemInline(
    admin.TabularInline
):
    model = OrderItem
    extra = 0

@admin.register(Order)
class OrderAdmin(admin.ModelAdmin):
    inlines = [OrderItemInline]

admin.site.register(OrderItem)

```

pytest.ini

INI

```

[pytest]
DJANGO_SETTINGS_MODULE = ai_box_selection.settings
python_files = tests.py test_*.py *_tests.py

```

tests/test_box_selector.py

Python

Run

```

import pytest

from shipping.models import Box
from shipping.services import BoxSelector

@pytest.mark.django_db
class TestBoxSelector:

    def setup_method(self):

        Box.objects.create(
            name="Small Box",
            length=10,
            width=10,
            height=10,
            max_weight=5,
            price=5
        )

```

```

Box.objects.create(
    name="Medium Box",
    length=20,
    width=20,
    height=20,
    max_weight=15,
    price=10
)

Box.objects.create(
    name="Large Box",
    length=40,
    width=40,
    height=40,
    max_weight=30,
    price=20
)

def test_select_smallest_cheapest_box(self):

    box = BoxSelector.recommend_box(
        length=5,
        width=5,
        height=5,
        weight=2
    )

    assert box.name == "Small Box"

def test_select_medium_box(self):

    box = BoxSelector.recommend_box(
        length=15,
        width=15,
        height=15,
        weight=10
    )

    assert box.name == "Medium Box"

def test_select_large_box(self):

    box = BoxSelector.recommend_box(
        length=35,
        width=35,
        height=35,
        weight=20
    )

    assert box.name == "Large Box"

```

```
def test_no_box_found(self):

    box = BoxSelector.recommend_box(
        length=100,
        width=100,
        height=100,
        weight=50
    )

    assert box is None
```

API Request Example

POST

http

POST /api/recommend-box/

Body

JSON

```
{
  "length": 15,
  "width": 10,
  "height": 8,
  "weight": 4
}
```

Response

JSON

```
{
  "box_name": "Medium Box",
  "price": "10.00"
}
```

Run Project

Bash

```
python manage.py makemigrations
python manage.py migrate
```

```
python manage.py createsuperuser
```

```
python manage.py runserver
```

Swagger UI:

```
http://localhost:8000/api/docs/
```

Schema:

```
http://localhost:8000/api/schema/
```

Run tests:

Bash

```
pytest
```

This implementation follows a clean architecture approach where the box recommendation logic is isolated in a dedicated `BoxSelector` service, making it easy to extend later with AI/ML-based recommendations, dimensional-weight calculations, carrier-specific box catalogs, or optimization algorithms.

